FILE PILE & ARBRE

1. File (Queue):

- ### Définition :

Une **file** est une **structure de type FIFO (First In, First Out)** : le premier élément ajouté est le premier retiré..

- ### Explication:

▢ Opérations principales :

1. enqueue() → ajouter un élément à la fin
2. dequeue() → retirer un élément au début

- ### Exemple :

```
#include <stdio.h>

#define MAX 5

int queue[MAX], front = 0, rear = -1;

void enqueue(int x) {

    if(rear == MAX-1) printf("File pleine\n");

    else queue[++rear] = x;

}

void dequeue() {

    if(front > rear) printf("File vide\n");
```

```
    else printf("Élément supprimé: %d\n", queue[front++]);
}
int main() {
    enqueue(10); enqueue(20); enqueue(30);
    dequeue(); dequeue(); dequeue(); dequeue();
    return 0;
}
```

2. Pile (Stack):

- ## Définition :

Une **pile** est une **structure de type LIFO (Last In, First Out)** : le dernier élément ajouté est le premier retiré.

- ## Explication:

- ☐ Opérations principales :
1. push() → ajouter un élément
2. pop() → retirer un élément
- ☐ Peut être implémentée avec **tableau ou liste chaînée**.

- ## Exemple :

```
#include <stdio.h>
#define MAX 5
int stack[MAX], top = -1;
void push(int x) {
    if(top == MAX-1) printf("Pile pleine\n");
    else stack[++top] = x;
}
```

```
void pop() {
    if(top == -1) printf("Pile vide\n");
    else printf("Élément supprimé: %d\n", stack[top--]);
}
int main() {
    push(10); push(20); push(30);
    pop(); pop(); pop(); pop();
    return 0;
}
```

3. Arbre (Tree):

- ## Définition :

Un **arbre** est une **structure hiérarchique composée de nœuds** :

- ☐ Chaque nœud a une **valeur** et **des pointeurs vers ses enfants**.
- ☐ Un **arbre binaire** est un arbre où chaque nœud a **au plus deux enfants** : gauche et droite.

- ## Explication:

- ☐ Le **nœud racine** est le point d'entrée.
- ☐ Permet **recherche, tri, hiérarchie et expressions arithmétiques**.

- ## Exemple :

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *left;
```

```
    struct Node *right;
};
struct Node* newNode(int data) {
    struct Node* node = (struct Node*) malloc(sizeof(struct Node));
    node->data = data;
    node->left = node->right = NULL;
    return node;
}
int main() {
    struct Node *root = newNode(10);
    root->left = newNode(20);
    root->right = newNode(30);
    printf("Racine: %d\n", root->data);
    printf("Gauche: %d, Droite: %d\n", root->left->data, root->right->data);
    return 0;
}
```